

Module 5

IEEE 1800-2009 and 1800-2012

Learning objectives

- Master the SystemVerilog revisions IEEE 1800-2009 and 1800-2012: interface and type refinements, op...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1800-2009 and 1800-2012"]

T2["Interface Refinements (2009)"]

T1 --> T2

T3["Type and Operator Enhancemen"]

T2 --> T3

Master the SystemVerilog revisions IEEE 1800-2009 and 1800-2012: interface and type refinements, operator and array enhancements, and asser...

Overview

- This module covers IEEE Std 1800-2009 and IEEE Std 1800-2012, the first two major revisions of Syst...

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics
Covered set the IEEE scope for Module 5.
- Study design architecture — See how DUTs, examples, and testbenches fit together under ``module5/``.
- Work through labs in order — Open ``module5/EXAMPLES.md`` and run each ``make clean && make run`` from...
- Run the full module script — ``./scripts/module5.sh`` from the repo root to simulate all examples and...
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at...

Suggested learning path (2/2)

- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use ``module5/CHECKLIST.md`` before starting the next module.

Syllabus topics

1. IEEE 1800-2009 and 1800-2012 Context (1/3)

- Interfaces: Modport expressions, virtual interface type (for testbenches), interface inheritance
- Types and operators: Clarifications; additional operators and built-in methods
- Assertions: SVA enhancements; checker concept (preview)
- DPI: Refinements for C/C++ interop
- Coverage: Coverage option and syntax clarifications
- Clarifications: Scheduling, generate, and other 2005 semantics

1. IEEE 1800-2009 and 1800-2012 Context (2/3)

- Checkers: Checker construct for reusable assertion/coverage logic
- Arrays: Array manipulation methods; array assignment and ordering clarifications
- Strings: String methods and behavior
- Compilation units: Refined rules for \$unit and compilation-unit scope
- Assertions and coverage: Further SVA and coverage features
- Clarifications: Numerous editorial and semantic clarifications across the LRM

1. IEEE 1800-2009 and 1800-2012 Context (3/3)

- 1800-2005 RTL remains valid in 1800-2009 and 1800-2012.
- New features are additive; you adopt them when the tool and project support the revision.

2. Interface Refinements (2009)

- Use: When the same interface is used with different signal subsets or derived signals; check tool s...
- Benefit: Fewer duplicate interface definitions; clearer connection contract.
- Design: Rarely used in RTL; RTL typically connects to a concrete interface instance.
- Verification: Common in UVM and other OOP testbenches; allows one driver class to work with any ins...

3. Type and Operator Enhancements (2009/2012)

(1/2)

- byte, shortint, int, longint: 8, 16, 32, 64 bits;
signed by default.
- byte unsigned, int unsigned: Unsigned variants.
- Clarifications in 2012: Representation and
conversion rules; use in RTL for constants and
parameter...
- inside: Already in 2005; 2009/2012 clarify set
membership and use with ranges.
- Wildcard equality `==?`: X/Z in the right-hand side
act as “don’t care” in comparisons.
- Set membership and case (inside): Used in case
statements and constraints; semantics clarified in
2...

3. Type and Operator Enhancements (2009/2012) (2/2)

- Array methods: ``size()`, ``delete()`, ``sort()`,
``reverse()`, etc. (exact set is revision-depende...
- Queues: ``push_back()`, ``pop_front()`, ``insert()`,
etc.
- Use in RTL: Limited (arrays in RTL are often fixed);
methods are common in testbenches and checkers.

4. Array and String Refinements (2012)

- Packed vs unpacked: Assignment rules and ordering (row-major, etc.) are clarified.
- Array copy: Whole-array assignment and copying; compatibility of dimensions.
- Use in RTL: Multi-dimensional arrays (e.g. memories) and parameterized dimensions; follow tool-supp...
- Strings: ``string`` type; methods such as ``.len()``, ``.substr()``, ``.atoi()``, etc. (as defined in 2012).
- Use in RTL: Rare (strings are mainly for messages and file I/O in testbenches).
- Use in testbenches: Logging, report generation, file names.

5. Assertions and Checkers in Context (2009/2012) (1/2)

- immediate assertions: ``assert (condition) else $error("...");`` — checked when executed (like an if-...
- concurrent assertions: ``assert property (@(posedge clk) (a |-> b));`` — checked over time; typically...
- Use in RTL: Immediate assertions for invariants (e.g. one-hot, valid encodings); concurrent asserti...
- checker: A reusable unit that can contain assertions, assumptions, coverage, and helper logic; can...
- Use: Encapsulate a set of properties (e.g. protocol rules) and reuse across modules or hierarchies.
- Design: Checkers can be bound to RTL modules; they do not change RTL functionality, only add checki...

5. Assertions and Checkers in Context (2009/2012) (2/2)

- Synthesis: Checkers are typically not synthesized; they are for simulation and formal tools.

6. Compilation Units and Scope (2012)

- Compilation unit: The scope of one or more source files compiled together (tool-defined).
- \$unit: The compilation-unit scope; declarations in \$unit are visible across the unit without import.
- Best practice: Prefer packages and explicit import over \$unit to avoid hidden dependencies and name...
- Import: ``import pkg::*`` or ``import pkg::item``; 2012 clarifies visibility and search order.
- Use: Keep shared types and functions in packages; import where needed; avoid relying on \$unit for n...

7. Summary: 1800-2005 vs 2009 vs 2012 (Design-Relevant)

- See docs/MODULE5.md

Hands-on examples

Module 5 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module5_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Example 1: Operators

- inside with ranges and sets; ==? wildcard (fallback for tools that lack inside)
- Set membership; don't-care in comparison

Demo: Operators

```
$ cd module5/examples/operators && make clean && make run
```

Terminal: ex01_operators

```
rm -f top
iverilog -g2012 -o top inside_wildcard.sv
vvp top
inside and ==? (1800-2009/2012)
  op= 1 inside{1,2,5,10}=1
  op= 3 in_set=0 mask==? 1x0z=x
  op= 5 in_set=1
inside_wildcard.sv:32: $finish called at 30 (1s)
```


Example 2: Wildcard Only

- Dedicated ==? wildcard equality (X/Z as don't care)
- Wildcard match in RTL/testbench

Demo: Wildcard Only

```
$ cd module5/examples/wildcard_only && make clean && make run
```

Terminal: ex02_wildcard_only

```
rm -f top
iverilog -g2012 -o top wildcard_only.sv
vvp top
Wildcard ==? (1800-2009/2012): X/Z on RHS = don't care
a=1000 pattern=1x0z match=1
a=1101 pattern=1x0z match=1
a=0100 pattern=1x0z match=0
wildcard_only.sv:28: $finish called at 30 (1s)
```

Example 3: Range Check

- Range membership (e.g. value inside [0:127]);
 fallback to explicit comparison if needed
- ``inside`` with ranges; 2009/2012 operator use

Demo: Range Check

```
$ cd module5/examples/range_check && make clean && make run
```

Terminal: ex03_range_check

```
rm -f top
iverilog -g2012 -o top range_check.sv
vvp top
Range check (1800-2009/2012): val inside [0:127] style
  val= 50 in_range=1
  val=127 in_range=1
  val=200 in_range=0
range_check.sv:26: $finish called at 30 (1s)
```

Example 4: Arrays

- Fixed memory with parameterized DEPTH/WIDTH; 2012 array semantics
- Array indexing; synthesis-safe subset

Demo: Arrays

```
$ cd module5/examples/arrays && make clean && make run
```

Terminal: ex04_arrays

```
rm -f top
iverilog -g2012 -o top array_methods.v
vvp top
Array (1800-2012): fixed array; parameterized DEPTH/WIDTH
  addr=3 dout=a3
array_methods.v:42: $finish called at 56 (1s)
```

Example 5: Array Param

- Parameterized memory using `$clog2(DEPTH)` for address width
- Parameterized arrays; 2012 clarifications

Demo: Array Param

```
$ cd module5/examples/array_param && make clean && make run
```

Terminal: ex05_array_param

```
rm -f top
iverilog -g2012 -o top array_param.sv
vvp top
Parameterized array (1800-2012): DEPTH=16 WIDTH=8 $clog2(DEPTH)=4
  addr=5 dout=55
array_param.sv:42: $finish called at 56 (1s)
```


Example 6: Checkers

- Immediate assertion (e.g. one-hot) using \$countones or similar
- assert in design; immediate assertion in RTL

Demo: Checkers

```
$ cd module5/examples/checkers && make clean && make run
```

Terminal: ex06_checkers

```
rm -f top
iverilog -g2012 -o top immediate_assert.sv
vvp top
Immediate assertion (1800-2009/2012): assert ($countones(sig) <= 1)
  sig=0001 valid=1
  sig=0100 valid=1
ERROR: immediate_assert.sv:13: onehot_check: sig not one-hot or zero: 0011
  Time: 20 Scope: top.u_check
  sig=0011 valid=0 (assert may fire)
immediate_assert.sv:28: $finish called at 30 (1s)
immediate_assert.sv:13: warning: System task ($error) cannot be synthesized in an always_comb pr
```

Example 7: Assert Valid Encoding

- Immediate assertion for valid opcode/encoding
- Invariant checking; assert for valid encodings

Demo: Assert Valid Encoding

```
$ cd module5/examples/assert_valid_encoding && make clean && make run
```

```
Terminal: ex07_assert_valid_encoding
rm -f top
iverilog -g2012 -o top assert_valid_encoding.sv
vvp top
Immediate assertion for valid encoding (1800-2009/2012)
  op=0 y= 1
  op=2 y= 4
ERROR: assert_valid_encoding.sv:14: alu_op: invalid opcode 5
  Time: 20 Scope: top.u_alu
  op=5 y= 0 (assert may fire)
assert_valid_encoding.sv:40: $finish called at 30 (1s)
assert_valid_encoding.sv:14: warning: System task ($error) cannot be synthesized in an always_co
```

Example 8: Assert Invariant

- Immediate assertion for one-hot grant (e.g. arbiter)
- One-hot invariant; assert in RTL

Demo: Assert Invariant

```
$ cd module5/examples/assert_invariant && make clean && make run
```

Terminal: ex08_assert_invariant

```
rm -f top
iverilog -g2012 -o top assert_invariant.sv
vvp top
Immediate assertion for invariant (1800-2009/2012): grant one-hot
  req=0100 grant=0100
  req=0011 grant=0010 (priority)
assert_invariant.sv:35: $finish called at 20 (1s)
assert_invariant.sv:10: sorry: constant selects in always_* processes are not currently supported
assert_invariant.sv:10: sorry: constant selects in always_* processes are not currently supported
assert_invariant.sv:10: sorry: constant selects in always_* processes are not currently supported
assert_invariant.sv:10: sorry: constant selects in always_* processes are not currently supported
assert_invariant.sv:23: warning: System task ($error) cannot be synthesized in an always_comb process
```

Example 9: Summary

- Comparison of 2005 vs 2009/2012 features in a small design
- Backward compatibility; when to adopt new features

Demo: Summary

```
$ cd module5/examples/summary && make clean && make run
```

Terminal: ex09_summary

```
rm -f top
iverilog -g2012 -o top summary_2012.sv
vvp top
2005 vs 2009/2012: same mux; 2012 uses inside for set check
  sel=10 y05=04 y12=04
summary_2012.sv:58: $finish called at 10 (1s)
```


Common pitfalls

Assuming 2009/2012 Features in an 2005-Only Flow

- Mistake: Using virtual interface, checker, or 2012 array methods when the project or tool is 1800-2...
- Reality: Compile or elaboration errors; or undefined behavior.
- Correct: Confirm project standard and tool revision; use only features of that revision.
- Why: Backward compatibility is 2005→2009→2012; tools may not implement all newer features.

Virtual Interface in RTL

- Mistake: Using a virtual interface in synthesizable RTL.
- Reality: Virtual interfaces are for dynamic binding (testbench); not for hardware.
- Correct: Use concrete interface instances in RTL; use virtual interface only in testbench classes.
- Why: Synthesis does not support virtual interface; it is a simulation construct.

Checker Synthesis

- Mistake: Expecting a checker to be synthesized like a module.
- Reality: Checkers are for assertion/coverage; typically not synthesized.
- Correct: Use checkers for verification; keep RTL in modules.
- Why: Checker semantics are for checking, not for netlist generation.

Array Methods in RTL

- Mistake: Using dynamic array or queue methods (e.g. `.push_back`) in synthesizable RTL.
- Reality: Many such methods are not synthesizable or are tool-specific.
- Correct: In RTL use fixed-size or parameterized arrays and indexing; use methods in testbenches or...
- Why: Synthesis subset of SystemVerilog is smaller than the full language.

\$unit vs Package

- Mistake: Putting shared types or functions in \$unit and relying on compilation order.
- Reality: \$unit visibility and order are tool-dependent; can cause subtle bugs.
- Correct: Put shared items in packages; use explicit import.
- Why: Packages give a clear namespace and import order; 2012 clarifies compilation unit but packages...

Practice & assessment

What you should know

- ✓ Describe what 1800-2009 and 1800-2012 add or clarify over 1800-2005 (interfaces, types, arrays, a...
- ✓ Use interface refinements (modport expressions, virtual interface) when the project and tool supp...
- ✓ Use ``inside`` and ``==?`` appropriately in RTL or testbenches
- ✓ Apply array and string refinements (2012) in testbenches or checkers; know the synthesizable subs...
- ✓ Use immediate assertions in RTL for simple invariants; understand checker (2012) as reusable prop...
- ✓ Summarize 1800-2005 vs 2009 vs 2012 and choose a

Exercises

- inside and ==?
- Interface (2009)
- Immediate Assertion
- Checker (2012)
- Version Table

Exercises

- Assuming 2009/2012 Features in an 2005-Only Flow
- Virtual Interface in RTL
- Checker Synthesis
- Array Methods in RTL
- \$unit vs Package

Summary & next steps

- Master the SystemVerilog revisions IEEE 1800-2009 and 1800-2012: interface and type refinements, op...
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: Next module in course